

C Compilers and Optimization

Introduction:

The aim of this module is to help you write C code in a style that will compile efficiently on the ARM architecture. We will look at many small examples to show how the compiler translates C source to ARM assembly. Once you have a feel for this translation process, you can distinguish fast C code from slow C code.

Overview of C Compilers and Optimization:

This chapter assumes that you are familiar with the C language and have some knowledge of assembly programming. The latter is not essential, but is useful for following the compiler output examples.

The optimizing code takes time and reduces source code readability. C compilers have to translate your C function literally into assembly so that it works for all possible inputs.

The memclr function clears N bytes of memory at address data.

```
void memclr (char *data, int N)
```

```
{  
    for (; N > 0; N--)
```

```
{  
        *data = 0;  
        data++;
```

```
}
```

```
}
```

No matter how advanced the compiler, it does not know whether N can be '0' on input or not. Therefore the compiler needs to test for this case explicitly before the first iteration of the loop.

Basic C Data Types :

Let's start by looking at how ARM compilers handle the basic C data types. we will see that some of these types are more efficient to use for local variables than others. There are also differences between the addressing modes available when loading and storing data of

(2)

Pothi Reddy.K

Asst. Prof, Dept of CSE, CITNC

each type.

ARM processors have 32-bit registers and 32-bit data processing operations. The ARM architecture is a RISC load/store architecture.

Early versions of the ARM architecture (ARM V1 to ARM V3) provided hardware support for loading and storing unsigned 8-bit and unsigned or signed 32-bit values.

Architecture	Instruction	Action
Pre-ARMV4	LDRB	Load an unsigned 8-bit value
	STRB	Store a signed or unsigned 8-bit value
	LDR	Load a signed or unsigned 32-bit value.
	STR	Store a sig or unsig. 32-bit value
ARMV4	LDRSB	Load a sig. 8-bit value
	LDRH	Load an unsig. 16-bit value
	LDRSH	Load a sig. 16-bit value
	STRH	Store a sig. or unsig 16-bit value
ARMV5	LDRD	Load a sig. or unsig. 64-bit value
	STRD	Store a sig or unsig 64-bit value.

Table ① : Load and store instructions by ARM architecture

These architectures were used on processors prior to the ARM7TDMI.

Prior to ARMV4, ARM processors were not good at handling signed 8-bit or any 16-bit values. Therefore ARM C Compilers define char to be an unsigned 8-bit value, rather than a signed 8-bit value as is typical in many other compilers.

Compilers armcc and gcc use the datatype mappings in Table ②, for an ARM target.

C Data Type	Implementation
char	unsigned 8-bit
short	signed 16-bit
int	signed 32-bit
long	signed 32-bit
long long	signed 64-bit

Table ②: C Compiler datatype mappings.

(i) Local Variable Types:

ARMv4-based processors can efficiently load and store 8, 16 and 32 bit data. However, most ARM data processing operations are 32-bit only. For this reason, you should use a 32-bit datatype, int or long, for local variables whenever possible. Avoid using char and short as local variable types, even if you are manipulating an 8 or 16 bit value.

To see the effect of local variable types, let's consider a simple example. We will look in detail at the checksum function that sums the values in a ~~data~~ data packet. Most communication protocols (such as TCP/IP) have a checksum or cyclic-redundancy check (CRC) routine to check for errors in a data packet.

The following code checksums a data packet containing 64 words. It shows why you should avoid using char for local variables.

```
int checksum_v1(int *data)
{
    char i;
    int sum = 0;
```

```
for (i = 0 ; i < 64 ; i++)
```

```
{  
    sum += data[i];
```

```
}
```

```
return sum;
```

```
}
```

At first sight it looks as though declaring 'i' as a char is efficient. You may be thinking that a char uses less register space or less space on the ARM stack than an int. On the ARM, both these assumptions are wrong. All ARM registers are 32-bit and all stack entries are at least 32-bit. Furthermore, to implement the `i++` exactly, the compiler must account for the case when `i = 255`. Any attempt to increment 255 should produce the answer '0'.

Consider the compiler output for this function. We've added labels and comments to make the assembly clear.

checksum_v1

MOV r2, r0 ; r2 = data

MOV r0, #0 ; sum = 0

MOV r1, #0 ; i = 0

checksum_v1_loop

LDR r3, [r2, r1, LSL #2] ; r3 = data[i]

ADD r1, r1, #1 ; r1 = i + 1

AND r1, r1, #0xff ; i = (char)r1

CMP r1, #0x40 ; Compare i, 64

ADD r0, r3, r0 ; sum += r3

BCC checksum_v1_loop ; if (i < 64) loop

MOV PC, r14 ; return sum

Now Compare this to the Compiler output where instead we declare 'i' as an unsigned int.

checksum_v2

MOV r2, r0 ; r2 = data

MOV r0, #0 ; sum = 0

MOV r1, #0 ; i = 0

checksum_v2_loop

LDR r3, [r2, r1, LSL #2] ; r3 = data[i]

ADD r1, r1, #1 ; r1 ++

CMP r1, #0x40 ; Compare i, 64

ADD r0, r3, r0 ; sum += r3

BCC checksum_v2_loop ; if (i < 64) goto loop

MOV PC, r14 ; return sum

⑦

(ii) Function Argument Types:

The Converting local variables from types char or short to type int increases performance and reduces code size. The same holds for function arguments. Consider the following simple function, which adds two 16-bit values, halving the second, and returns a 16-bit sum.

```
Short add-VI (short a, short b)
{
    return a + (b >> 1);
}
```

This function is a little artificial, but it is a useful test case to illustrate the problems faced by the compiler. The input values a, b and the return value will be passed in 32-bit ARM-registers.

The armcc output for add-VI shows that the compiler casts the return value to a short type, but does not cast the input values. It assumes that the caller has already ensured that the 32-bit values r0 and r1 are in the range of the short type.

add_v1

```
ADD r0, r0, r1, ASR #1 ; r0 = (int)a +  
                        ((int)b >> 1)
```

```
MOV r0, r0, LSL #16
```

```
MOV r0, r0, ASR #16 ; r0 = (short)r0
```

```
MOV PC, r14 ; return r0
```

The gcc compiler we used is more cautious and makes no assumptions about the range of argument value.

(iii) Signed Versus Unsigned Types :

The advantages of using int rather than a char or short type for local variables and function arguments. This section compares the efficiencies of signed int and unsigned int.

If your code uses addition, subtraction and multiplication, then there is no performance difference between signed and unsigned operations. However, there is a difference when it comes to division. Consider the following short example that averages two integers :

```

int average_V1 (int a, int b)
{
    return (a+b)/2 ;
}

```

this compiles to

average_V1

```

ADD r0, r0, r1 ; r0 = a+b
ADD r0, r0, r0, LSR #31 ; if (r0 < 0) r0++
MOV r0, r0, ASR #1 ; r0 = r0 >> 1
MOV PC, r14 ; return r0

```

C Looping Structures :

The most efficient ways to code for and while loops on the ARM. we start by looking at loops with a fixed number of iterations and then move on to loops with a variable number of iterations.

(i) Loops with a Fixed number of Iterations:

The most efficient way to write a for loop on the ARM, let us return to our checksum example and look at the looping structure.

The Compiler treats a loop with incrementing count $i++$.

```
int checksum_v5 (int *data)
{
    unsigned int i;
    int sum = 0;
    for (i = 0 ; i < 64 ; i++)
    {
        sum += *(data++);
    }
    return sum;
}
```

this compiles to

checksum_v5

mov r2, r0 ; r2 = data

mov r0, #0 ; sum = 0

MOV $r_1, \#0$; $i = 0$

Checksum_V5_loop

LDR $r_3, [r_2], \#4$; $r_3 = * (data++)$

ADD $r_1, r_1, \#1$; $i++$

CMP $r_1, \#0x40$; Compare $i, 64$

ADD r_0, r_3, r_0 ; $Sum += r_3$

BCC Checksum_V5_loop ; if ($i < 64$) goto loop

MOV PC, r_14 ; return Sum

It takes three instructions to implement the for loop structure :

- An ADD to increment i
- A Compare to check if i is less than 64
- A Conditional branch to continue the loop if $i < 64$

This is not efficient. On the ARM, a loop should only use two instructions :

- A Subtract to decrement the loop counter, which also sets the Condition Code flags on the result.
- A Conditional branch instruction.

The Key point is that the loop Counter should Count down to zero rather than Counting up to some arbitrary limit. Then the comparison with zero is free since the result is stored in the condition flags. Since we are no longer using 'i' as an array index, there is no problem in Counting down rather than up.

The improvement if we switch to a decrementing loop rather than an incrementing loop.

```
int checksum_v6(int *data)
```

```
{
```

```
    unsigned int i;
```

```
    int sum = 0;
```

```
    for (i = 64; i != 0; i--)
```

```
    {
```

```
        sum += *(data++);
```

```
    }
```

```
    return sum;
```

```
}
```

This Compiles to

checksum_v6

MOV r2, r0 ; r2 = data

MOV r0, #0 ; Sum = 0

MOV r1, #0x40 ; i = 64

checksum_v6_loop

LDR r3, [r2], #4 ; r3 = *(data++)

SUBS r1, r1, #1 ; i--

ADD r0, r3, r0 ; Sum += r3

BNE checksum_v6_loop ; if (i != 0) goto loop

MOV PC, r14 ; return Sum.

(ii) Loops Using a Variable Number of Iterations:

Now suppose we want our checksum routine to handle packets of arbitrary size. we pass in a variable N giving the number of words in the data packet. we count down until $N=0$ and don't require an extra loop counter i . The Compiler handles a for loop with a variable number of iterations N .

```

int checksum_v7 (int *data, unsigned int N)
{
    int sum = 0;
    for (; N != 0; N--)
    {
        sum += *(data++);
    }
    return sum;
}

```

This compiles to

```

checksum_v7
    mov    r2, #0 ; sum = 0
    cmp    r1, #0 ; compare N, 0
    beq    checksum_v7_end ; if (N == 0) goto
                                end
checksum_v7_loop
    ldr    r3, [r0], #4 ; r3 = *(data++)
    subs   r1, r1, #1 ; N--
    add    r2, r3, r2 ; sum += r3
    bne    checksum_v7_loop ; if (N != 0) goto
                                loop
checksum_v7_end
    mov    r0, r2 ; r0 = sum
    mov    pc, r14 ; return r0

```

Notice that the compiler checks that N is non-zero on entry to the function. often this checks is necessary since you know that the array won't be empty. In this case a do-while loop gives better performance and code density than a for loop.

```
int checksum_v8(int *data, unsigned int N)
{
    int sum = 0;
    do
    {
        sum += *(data++);
    } while (--N != 0);
    return sum;
}
```

The compiler output is now

checksum_v8

MOV r2, #0 ; sum = 0

checksum_v8_loop

LDR r3, [r0], #4 ; r3 = *(data++)

SUBS r1, r1, #1 ; N--

ADD r2, r3, r2 ; sum += r3

```
BNE checksum_v8_loop ; if(N!=0) goto loop
MOV r10, r12           ; r10 = sum
MOV PC, r14            ; return r10
```

Register Allocation:

The Compiler attempts to allocate a processor register to each local variable you use in a 'C' function. It will try to use the same register for different local variables if the use of the variables do not overlap. When there are more local variables than available registers, the Compiler stores the excess variables on the processor stack. These variables are called spilled or swapped out variables. Spilled variables are slow to access compared to variables allocated to registers.

To implement a function efficiently, you need to, (i) minimize the number of spilled variables

(ii) The most important and frequently accessed variables are stored in registers.

the number of processor registers the ARM C Compilers have available for allocating variables. The Table ③, shows the standard register names and usage when following the ARM-Thumb procedure call-standard (ATPCS), which is used in code generated by C Compilers.

Register number	Alternative register names	ATPCS register usage
r_0 r_1 r_2 r_3	a_1 a_2 a_3 a_4	Argument registers. These hold the first four function arguments on a function call and the return value on a function return.
r_4 r_5 r_6 r_7 r_8	v_1 v_2 v_3 v_4 v_5	General variable registers. The function must preserve the callee values of these registers.

r_9	V_6 Sb	The R_9 holds the static base address.
r_{10}	V_7 SL	The R_{10} holds the stack limit address.
r_{11}	V_8 FP	The function must preserve the callee value of this register except when compiling using a frame pointer.
r_{12}	IP	It is useful as a scratch-register for function veneers or other intraprocedure call requirements.
r_{13}	SP	The stack pointer, pointing to the full descending stack.
r_{14}	LR	The Link Register, on a function call this holds the return-address.
r_{15}	PC	the program counter.

Function Calls:

The ARM Procedure Call Standard (APCS) defines how to pass function arguments and return values in ARM registers. The more recent ARM-Thumb Procedure Call Standard (ATPCS) covers ARM and Thumb interworking as well.

The first four integer arguments are passed in the first four ARM registers: r_0, r_1, r_2 and r_3 . Subsequent integer arguments are placed on the full descending stack, ascending in memory as in below figure ①. Function return integer values are passed in r_0 .

...	...
$SP+16$	Argument 8
$SP+12$	Argument 7
$SP+8$	Argument 6
$SP+4$	Argument 5
SP	Argument 4

r_3	Argument 3	
r_2	Argument 2	
r_1	Argument 1	
r_0	Argument 0	Return Value

②①

Figure ①. ATPCS argument passing

This description covers only integer or pointer arguments. Two-word arguments such as long long (or) double are passed in a pair of consecutive argument registers and returned in r_0, r_1 .

Example:-

This example illustrates the benefits of using a structure pointer. First we show a typical routine to insert 'N' bytes from array data into a queue. we implement the queue using a cyclic buffer with start address Q_start (inclusive) and end address Q_end (exclusive).

Char * queue_bytes_vl (

Char * Q_start, /* Queue buffer start address */
Char * Q_end, /* " " " " end " */
Char * Q_ptr, /* current queue pointer position */
Char * data, /* Data insert into the queue */
unsigned int N) /* No. of bytes to insert */

```
{  
    do  
    {  
        *(Q_ptr++) = *(data++);  
        if (Q_ptr == Q_end)  
        {
```

```

    {
        Q_ptr = Q_start ;
    }

} while(--N);
return Q_ptr;
}

```

This Compiles to

queue_bytes_V1

```

STR r14, [r13, #-4]! ; save LR on the stack
LDR r12, [r13, #4] ; r12 = N

```

queue_V1_loop

```

LDRB r14, [r13], #1 ; r14 = *(data++)
STRB r14, [r12], #1 ; (Q_ptr++) = r14
CMP r2, r1 ; if (Q_ptr == Q_end)
MOVEQ r2, r0 ; { Q_ptr = Q_start; }
SUBS r12, r12, #1 ; --N
BNE queue_V1_loop ; if (N != 0) goto loop
MOV r0, r2 ; r0 = Q_ptr
LDR PC, [r13], #4 ; return r0.

```

Pointer Aliasing :

Two pointers are said to alias when they point to the same address. If you write to one pointer, it will affect the value you read from the other pointer. In a function, the compiler often doesn't know which pointers can alias and which pointers can't.

Example :-

The following function increments two-timer values by a step amount :

```
void timers_v1(int *timer1, int *timer2, int *step)
{
    *timer1 += *step;
    *timer2 += *step;
}
```

This compiles to

timers_v1

LDR r3, [r0, #0] ; r3 = *timer1

LDR r12, [r2, #0] ; r12 = *step

ADD r3, r3, r12 ; r3 += r12

STR r3, [r0, #0] ; *timer1 = r3

LDR r0, [r1, #0] ; r0 = *timer2

LDR r2, [r2, #0] ; r2 = *step

ADD r_0, r_0, r_2 ; $r_0 + = r_2$

STR $r_0, [r_1, \#0]$; $*timer_1 = r_0$

MOV PC, r_{14} ; return

The Compiler loads from `step` twice.

Usually a Compiler optimization called Common-Subexpression elimination would kick in so that `*step` was only evaluated once, and the value reused for the second occurrence. However, the Compiler can't use this optimization here. The pointers `timer_1` and `step` might alias one another.

In other words, the Compiler cannot be sure that the write to `timer_1` doesn't affect the read from `step`. In this case the second value of `*step` is different from the first and has the value `*timer_1`. This forces the Compiler to insert an extra load instruction.

Portability Issues :

Here is a summary of the issues you may encounter when porting C code to the ARM.

i) The char type :

On the ARM, char is unsigned rather than signed as for many other processors. A common problem concerns loops that use a char loop counter "i" and the continuation condition $i \geq 0$, they become infinite loops. In this situation, armcc produces a warning of unsigned comparison with zero. You should either use a compiler option to make char signed or change loop counters to type int.

ii) The int type :

Some older architectures use a 16-bit int, which may cause problems when moving to ARM's 32-bit int type although this is rare nowadays. Note that expressions are promoted to an int type before evaluation.

iii) Unaligned data pointers:

Some processors support the loading of short and int typed values from unaligned - addresses. A C program may manipulate pointers directly, so that they become unaligned.

iv) Endian assumptions:

C code may make assumptions about the endianness of a memory system, for example, by casting a char * to an int *. If you configure the ARM for the same endianness the code is expecting, then there is no issue. otherwise, you must remove endian - dependent code sequences and replace them by endian - independent ones.

v) Function Prototyping:

The armcc compiler passes arguments narrow, that is, reduced to the range of the argument type. If functions are not prototyped correctly, then the function may return the wrong answer. other compilers that pass arguments wide may give the correct answer even if the function prototype is incorrect. Always use ANSI prototypes.

vi) Use of bit-fields:

The layout of bits within a bit-field is implementation and endian dependent. If C code assumes that bits are laid out in a certain order, then the code is not portable.

X ← END → X